

Chapter 4

Syntax Directed Definition

Syntax Directed Definition:

1. We can associate information with a language construct by attaching attributes to the grammar symbols.
2. We can associate information with a language construct by attaching attributes to the grammar symbols.
3. A SDD is a context free grammar with attributes and rules.
4. Attributes are associated with grammar symbols and rules with productions.
5. Attributes may be of many kinds: numbers, types, table references, strings, etc.
6. Synthesized attributes: A synthesized attribute at node N is defined only in terms of attribute of N and at N itself.
7. Inherited attributes: An inherited attribute at node N is defined only in terms of attribute values at N's parent, N itself and N's siblings.

Synthesized Attributes

1. A synthesized attribute at node N is defined only in terms of attribute values of children of N and at N itself.
2. Each of the non-terminals has a single synthesized attribute, called val.
3. An SDD that involves only synthesized attributes is called S-attributed.
4. Each rule computes an attribute for the non-terminal at the head of a production from attributes taken from the body of the production.

Production	Semantic Rules
$T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

1. A
parse
tree,
show
ing
the
value
(s) of
its
attrib

- ute(s) is called an annotated parse tree.
2. A parse tree, showing the value(s) of its attribute(s) is called an annotated parse tree.

Inherited Attributes

1. An INHERITED ATTRIBUTE for a non-terminal B at a parse-tree node N is defined by a semantic rule associated with the production at the parent of N. The production must have B as a symbol in its body.
2. An inherited attribute at node N is defined only in terms of attribute values at N's parent, N itself, and N's siblings.

Example:

	PRODUCTION	SEMANTIC RULE
	$D \rightarrow TL$	$L.in := T.type$
Evaluation Order	$T \rightarrow \text{int}$	$T.type := \text{integer}$
	$T \rightarrow \text{real}$	$T.type := \text{real}$
	$L \rightarrow L_1, \text{id}$	$L_1.in := L.in; \text{ addtype}(\text{id.entry}, L.in)$
	$L \rightarrow \text{id}$	$\text{addtype}(\text{id.entry}, L.in)$

s for SDD's

- Dependency graphs“ tool for determining an evaluation order for the attribute instances in a given parse tree.
- annotated parse tree shows the values of attributes, a dependency graph helps to determine how those values can be computed.
- Edges express constraints implied by the semantic rules.
- Each attribute is associated to a node
- A dependency graph characterizes the possible order in which we can evaluate the attributes at various nodes of a parse tree.
- If there is an edge from node M to N, then attribute corresponding to M first be evaluated before evaluating N.
- If there is any cycle in the graph, then there are no topological sorts.

S-Attributed

- 1.If every attribute is synthesized.
2. S-attributed SDD can be evaluated in bottom up order of the nodes of the parse tree.

L-Attributed

- Synthesized, or Inherited, but with the rules limited as follows. Suppose that there is a production $A \rightarrow X_1X_2 \dots X_n$, and that there is an inherited attribute X_i computed by a rule associated with this production.

- Inherited attributes associated with the head A.
- Either inherited or synthesized attributes associated with the occurrences of symbols $X_1, X_2, \dots, X_{i-1}$ located to the left of X_i .
- Inherited or synthesized attributes associated with this occurrence of X_i itself, but only in such a way that there are no cycles in a dependency graph formed by the attributes.

Recursive Descent Parser

- Top-down parsing strategy, for LL(1) grammars.
- One procedure per nonterminal.
- Stack contents embedded in recursive call sequence.
- Each procedure “commits” to one production, based on the next input symbol, and the select sets.
- It uses back- tracking. So, it is not usefull.